

Students Graduation Clearance System (SGCS)

Technical Presentation & Defense Notes

Prepared from the supplied SGCS project notes

1. System overview

Simple presentation explanation

SGCS is a web-based Students Graduation Clearance System for Tangaza University. It replaces a manual clearance process where a graduating student would move documents between offices.

A student uploads a final transcript and starts a clearance request. The request then moves in a fixed order through:

Student submits transcript



Institute Director



Library



Finance Office



Dean of Students



University Store



Academic Registrar



FINAL APPROVAL

Each office can approve or reject the request. The student can track progress, see comments, upload a corrected transcript after rejection, and finally view a Final Approval screen.

Users are:

- Student: creates requests, uploads documents, tracks status, sees Final Approval.
- Institute Director: first reviewer.
- Library Officer: reviews the request and can view the final-year-project document if uploaded.
- Finance Officer: reviews the request and can view a fee statement if uploaded.
- Dean of Students: reviews student-affairs clearance.

- University Store Officer: reviews store-related clearance.
- Academic Registrar: final reviewer; their approval creates Final Approval.
- System Administrator: manages user accounts, account status, passwords, and officer-to-department assignment.

Academic Programme Leader is not in the implemented workflow.

Technical explanation

This is a server-rendered PHP/MySQL application using PDO. PHP pages receive form submissions, validate inputs, execute prepared SQL statements through PDO, then render HTML pages. Authentication is PHP-session based. The workflow state is represented mainly by:

- ``clearance_requests.current_dept_id``
- ``clearance_requests.overall_status``
- ``approval_records.status``
- ``departments.sequence_no``

The central workflow method is `update_clearance_decision()` in `[clearance\_functions.php](C:/Users/K RICH/OneDrive/Music 2.0/SGCS/SGCS/includes/clearance\_functions.php)`.

2. Project structure

Folder/file	Purpose
<code>`config/`</code>	Application constants and PDO database connection.
<code>`includes/`</code>	Shared authentication, business/workflow functions, header, and footer.
<code>`dashboards/`</code>	Different landing dashboards for students, officers, director, registrar, and admin.
<code>`database/schema.sql`</code>	Creates tables, relationships, demo accounts, workflow departments, and seed data.
<code>`uploads/`</code>	Stores uploaded transcript files; code can also create <code>`uploads/documents/`</code> for supporting documents.
<code>`assets/css/style.css`</code>	Visual styling and responsive layout.
<code>`assets/js/main.js`</code>	Small JavaScript enhancement for alert accessibility.
<code>`officer_requests.php`</code>	Shared approval/rejection page for departmental users.
<code>`student_clearance_request.php`</code>	Student starts or resubmits clearance.
<code>`final_approval.php`</code>	Student-only, view-only Final Approval page.
<code>`admin_users.php`</code>	Admin account management.

Most important files:

File	Purpose and connections
<code>[config.php](C:/Users/K RICH/OneDrive/Music 2.0/SGCS/SGCS/config/config.php)</code>	Defines app name, URL, database name <code>`sgcs_db`</code> , and session name <code>`sgcs_session`</code> .
<code>[database.php](C:/Users/K RICH/OneDrive/Music 2.0/SGCS/SGCS/config/database.php)</code>	Creates <code>\$pdo</code> , the PDO connection used by all database operations.
<code>[auth.php](C:/Users/K RICH/OneDrive/Music 2.0/SGCS/SGCS/includes/auth.php)</code>	Starts sessions and provides <code>`is_logged_in()`</code> , <code>`current_user()`</code> , <code>`require_login()`</code> , and <code>`require_role()`</code> .
<code>[clearance_functions.php](C:/Users/K RICH/OneDrive/Music 2.0/SGCS/SGCS/includes/clearance_functions.php)</code>	Main business logic: uploads, request creation, workflow movement, notifications, audit logs, and reports.
<code>[login.php](C:/Users/K RICH/OneDrive/Music 2.0/SGCS/SGCS/login.php)</code>	Validates credentials and stores the user identity/role in the session.
<code>[student_clearance_request.php](C:/Users/K RICH/OneDrive/Music 2.0/SGCS/SGCS/student_clearance_request.php)</code>	Calls <code>`start_clearance_request()`</code> for a new request or transcript resubmission functions after rejection.
<code>[officer_requests.php](C:/Users/K RICH/OneDrive/Music 2.0/SGCS/SGCS/officer_requests.php)</code>	Lists only requests currently assigned to the logged-in officer's department and calls <code>`update_clearance_decision()`</code> .
<code>[final_approval.php](C:/Users/K RICH/OneDrive/Music 2.0/SGCS/SGCS/final_approval.php)</code>	Displays a final approval only to the student whose latest request

2.0/SGCS/SGCS/final_approval.php)	has status `Final`.
[admin_users.php](C:/Users/K RICH/OneDrive/Music 2.0/SGCS/SGCS/admin_users.php)	Creates, activates/deactivates, soft-deletes, resets users, and may assign a new account to a department.

3. Database

Database name: sgcs_db, defined in [config.php](C:/Users/K RICH/OneDrive/Music 2.0/SGCS/SGCS/config/config.php) and created in [schema.sql](C:/Users/K RICH/OneDrive/Music 2.0/SGCS/SGCS/database/schema.sql).

Simple explanation

The database separates people, their accounts, departments, clearance requests, approval decisions, documents, notifications, final approvals, and audit history. This avoids repeatedly storing the same student or department data in every clearance record.

Key tables and relationships

Table	Key purpose
`students`	Student academic/profile information. PK: `student_id`.
`users`	Login accounts. PK: `user_id`; stores username, password hash, role, and `linked_id`.
`departments`	Workflow offices and their order. PK: `dept_id`; `sequence_no` determines sequence.
`clearance_requests`	One submitted clearance process. PK: `request_id`; links student and current department.
`approval_records`	One decision record per request/department. PK: `approval_id`; stores status, officer, comments, and dates.
`transcripts`	Final transcript upload records.
`supporting_documents`	Final-year-project and fee-statement uploads.
`notifications`	In-app messages for users and officers.
`final_approvals`	Final approval record; one per request due to `request_id` UNIQUE`.
`audit_logs`	History of important actions.

Important relationships:

students 1 — many clearance_requests

clearance_requests 1 — many approval_records

departments 1 — many approval_records

users 1 — many notifications

clearance_requests 1 — one final_approvals

clearance_requests 1 — many transcripts/supporting_documents

users.linked_id connects a student login account to students.student_id. For staff, departments.officer_user_id connects a department to its assigned officer.

Normalization

The schema is broadly normalized to approximately Third Normal Form:

- Student details are stored once in `students`.
- Credentials and roles are stored separately in `users`.

- Department definitions are separate from decisions.
- Each approval decision belongs to a specific request and department.

This is appropriate because it reduces duplication and makes workflow reporting easier. One limitation is that `users.linked_id` is a generic link rather than a database-enforced foreign key; it works for this project but is weaker than a dedicated student-user foreign key.

4. Login, authentication, and authorization

Login is implemented in `[login.php](C:/Users/K RICH/OneDrive/Music 2.0/SGCS/SGCS/login.php)`.

1. The user submits username and password.
2. ``clean_input()`` sanitizes the username.
3. A prepared query selects an active, non-deleted user:

```
SELECT * FROM users WHERE username = ? AND account_status = 'Active' AND deleted_at IS NULL
```

4. ``password_verify($password, $user['password_hash'])`` checks the entered password against the saved hash.
5. If valid, PHP session values are created:

```
$_SESSION['user_id']
$_SESSION['username']
$_SESSION['full_name']
$_SESSION['role']
$_SESSION['linked_id']
```

6. The user is redirected to `[dashboard.php](C:/Users/K RICH/OneDrive/Music 2.0/SGCS/SGCS/dashboard.php)`, which routes them according to role.
7. Logout in `[logout.php](C:/Users/K RICH/OneDrive/Music 2.0/SGCS/SGCS/logout.php)` clears the session, expires its cookie, destroys the session, and redirects to login.

Presentation answer:

Authentication answers “Who is this user?” The system checks their username and password. Authorization answers “What is this authenticated user allowed to do?” The system checks their role before allowing access to a page.

5. Role-based access control

RBAC is centralized in `[auth.php](C:/Users/K RICH/OneDrive/Music 2.0/SGCS/SGCS/includes/auth.php)`.

- ``require_login()`` redirects a visitor without ``$_SESSION['user_id']`` to ``login.php``.
- ``require_role($roles)`` ensures the logged-in role is in the allowed role list; otherwise it redirects to ``dashboard.php``.

Examples:

Role	Actual access
Student	Clearance request, transcript upload, supporting documents, tracking, profile, notifications, Final Approval.

Director	Assigned requests, approval history, department reports, notifications.
Library/Finance/Dean/Store Officer	Assigned requests, decision entry, approval history, notifications.
Registrar	Assigned final-stage requests, transcripts, reports, audit trail, notifications.
Admin	User management, reports, audit trail, notifications.

If a student manually enters `officer_requests.php`, `require_role(['officer', 'director', 'registrar'])` rejects access and redirects them to their dashboard.

Important nuance: the application has five stored role values—student, officer, director, registrar, and admin. Library, Finance, Dean, and Store are all officer accounts; their specific department comes from `departments.officer_user_id`.

6. Clearance request creation

The main path is:

```
student_clearance_request.php
→ start_clearance_request()
→ validate uploaded transcript
→ save file
→ begin transaction
→ create clearance request
→ create Director approval record
→ create notifications/audit logs
→ commit
```

`start_clearance_request()`:

- requires a successful upload;
- limits the file to 5 MB;
- checks MIME type using ``finfo`` and accepts only ``application/pdf``;
- finds the Institute Director through ``get_institute_director_department()``;
- saves the transcript as ``transcript_{student_id}_{timestamp}.pdf``;
- inserts into ``clearance_requests`` with status ``Pending``;
- inserts the initial Director ``approval_records`` row;
- inserts a ``transcripts`` record;
- notifies the Director and student;
- logs the request and transcript upload.

Why PDO transactions are used:

A clearance request is not one database action. It creates a request, approval record, transcript record, notifications, and audit records. A transaction makes them succeed together or fail together. If a database error occurs, ``rollBack()`` undoes the database changes.

The code also removes the saved transcript file if the database transaction fails. That is a good consistency measure.

7. File uploads

Implemented uploads

UploadFile/functionValidation		
Student final transcript	`start_clearance_request()` / `submit_transcript()`	PDF MIME type; maximum 5 MB.
Final Year Project	`upload_supporting_document()`	PDF MIME type; maximum 5 MB.
Fee statement	`upload_supporting_document()`	PDF MIME type; maximum 5 MB.
Registrar transcript upload	[registrar_transcripts.php](C:/Users/K RICH/OneDrive/Music 2.0/SGCS/SGCS/registrar_transcripts.php)	Extension `.pdf`; maximum 5 MB.

Files are stored in:

- `uploads/transcripts/`
- `uploads/documents/`

The database stores generated file name and relative file path. Officers view documents through links in [officer_requests.php](C:/Users/K RICH/OneDrive/Music 2.0/SGCS/SGCS/officer_requests.php).

Security explanation:

Upload validation helps prevent attackers from submitting executable or unexpected files. The student upload path checks actual MIME type, not only the file extension, which is stronger.

Actual weaknesses to mention honestly:

- Uploaded files are in web-accessible folders and document links do not re-check authorization; someone who learns a file URL may be able to access it directly.
- The registrar transcript upload checks extension only, not MIME type.
- There is no malware scanning.
- There is no per-file download authorization controller.
- Generated filenames use a timestamp, which is practical but not as collision-resistant as random cryptographic names.
- Supporting-document upload is not wrapped in a database transaction.

8. Workflow engine

The main function is `update_clearance_decision()` in [clearance_functions.php](C:/Users/K RICH/OneDrive/Music 2.0/SGCS/SGCS/includes/clearance_functions.php).

Request's `current_dept_id`

↓

Officer receives only matching pending requests

↓

Officer approves or rejects

↓

Approval record updated with officer, status, comments, `decided_at`

↓

If approved: next department is found by sequence_no

If rejected: request status becomes Rejected

↓

Notifications and audit log are recorded

How sequential control works:

8. `officer\_requests.php` selects only rows where:

```
cr.current_dept_id = officer_department
```

```
AND ar.status = 'Pending'
```

9. Before saving a decision, `update\_clearance\_decision()` confirms:

```
$request['current_dept_id'] === $dept_id
```

10. The next department is determined using the next higher `departments.sequence\_no`.

11. When the Director approves, the code creates pending approval records for all later departments:

```
INSERT INTO approval_records (request_id, dept_id, status)
```

```
SELECT ?, dept_id, 'Pending'
```

```
FROM departments
```

```
WHERE sequence_no > ?
```

12. Only one department is active at a time because `current\_dept\_id` changes only after approval.

Rejection:

- The current `approval\_records` row becomes `Rejected`.
- `comments` stores the reason.
- Overall request status becomes `Rejected`.
- The student receives `clearance\_rejected`.

Resubmission:

- Implemented.
- The student uploads a corrected transcript.
- `resubmit\_clearance\_request()` resets the rejected department's decision to `Pending`.
- It changes the request to `In Progress`, returns it to the rejected department, and notifies both student and officer.

9. Library module

There is no separate library.php; the Library Officer uses the shared [officer_requests.php](C:/Users/KRICH/OneDrive/Music 2.0/SGCS/SGCS/officer_requests.php).

The page:

- identifies the officer's assigned department through `get\_assigned\_department()`;

- lists only Library requests currently at Library;
- displays transcript link;
- retrieves the latest `final\_year\_project` file as `project\_path`;
- provides Approve and Reject buttons;
- requires a rejection comment at page level;
- calls `update\_clearance\_decision()`.

Important accuracy point:

The system allows Library to view a Final Year Project if uploaded, but it does not technically force a project upload or implement special project-evaluation rules. Approval remains a manual officer decision.

10. Finance module

Finance also uses the shared approval page.

The page retrieves the latest fee-statement upload as fee_path, displays it where available, and lets Finance approve or reject with comments.

Not implemented:

- Automatic graduation-fee calculation.
- Payment gateway integration.
- Automatic verification against a finance ledger or bank/payment system.
- Mandatory fee-statement upload before Finance approval.

Safe presentation answer:

In the current version, Finance verification is a controlled manual review. The system stores and displays a fee statement if the student uploads one, then the Finance Officer records the decision.

11. Academic Registrar and Final Approval

The Academic Registrar is final because Academic Registrar has sequence_no = 6, the last department in [schema.sql](C:/Users/K RICH/OneDrive/Music 2.0/SGCS/SGCS/database/schema.sql).

The Registrar cannot approve early because their request only appears when:

- previous officers approved;
- `current\_dept\_id` has been advanced to the Registrar;
- the Registrar's `approval\_records` row is pending.

When the Registrar approves, there is no next department. The code:

13. Sets `clearance\_requests.overall\_status` to `Final`.
14. Sets `current\_dept\_id` to `NULL`.
15. Inserts or updates `final\_approvals`.
16. Sends `final\_approval\_generated` notification.
17. Writes an audit-log entry.

What Final Approval contains

[final_approval.php](C:/Users/K RICH/OneDrive/Music 2.0/SGCS/SGCS/final_approval.php) displays:

Students Graduation Clearance System (SGCS)

- Approval number: ``final_approvals.approval_number``
- Student name
- Registration number
- Programme
- Issue date: ``generated_at``
- Status: ``FINAL APPROVED``
- Approved by: Academic Registrar

It is view-only because the page has no download, edit, or generation action; it merely reads existing data.

Not implemented:

- PDF generation.
- Downloadable certificate.
- A stored ``approved_by`` field in ``final_approvals``; “Academic Registrar” is inferred from the workflow’s final stage.
- A separate issue-date field; ``generated_at`` acts as the issue date.

Remaining certificate terminology:

- ``DROP TABLE IF EXISTS certificates;`` in ``schema.sql``.
- Several old ``student_certificate.php`` references in ``README.md``; that file does not exist in the present project.
- The current working feature itself consistently uses “Final Approval.”

12. Notifications

Notifications are stored in the notifications table and created through `add_notification()`.

Important events that create notifications:

- Student starts clearance: Director and student.
- Student uploads/resubmits transcript: responsible department and student.
- Officer approves: student and next department’s officer.
- Officer rejects: student.
- Registrar finalizes: student.
- Student requests transcript support: Registrar and student.
- Registrar uploads transcript: student.
- Student registers: administrator.

[`notifications.php`](C:/Users/K RICH/OneDrive/Music 2.0/SGCS/SGCS/notifications.php) retrieves only the logged-in user’s notifications using `recipient_user_id`. It displays read status. `?mark_read=1` marks all notifications for that user as read.

13. Admin module

Actual admin functionality in [`admin\_users.php`](C:/Users/K RICH/OneDrive/Music 2.0/SGCS/SGCS/admin_users.php):

- Create user accounts.
- Choose one of the five roles.

- Assign a newly created account to an existing department.
- Search/filter users.
- Activate/deactivate users.
- Reset a user password to literal demo value `password`.
- Soft-delete users using `account\_status = 'Inactive'` and `deleted\_at`.

Admin can also access reports, audit trail, and notifications.

Not implemented despite dashboard wording:

- Full student-record management.
- Creating, editing, deleting, or resequencing departments.
- Full role-definition management.
- Editing existing user details or reassigning an existing user to a different department through the UI.

14. Security and error handling

Implemented protections

Mechanism	Where	Protection
Password hashing	`password_hash()` in registration/admin/reset	Passwords are not stored in plain text.
Password verification	`password_verify()` in login	Verifies without revealing the stored password.
Prepared statements	Throughout `clearance_functions.php` and pages	Protects SQL queries from injection when parameters are used.
PDO exception mode	`database.php`	Converts database failures into exceptions that can be handled.
Session login checks	`require_login()`	Blocks unauthenticated pages.
Role checks	`require_role()`	Blocks role-inappropriate pages.
Output escaping	Widespread `htmlspecialchars()`	Reduces XSS risk when displaying stored/user-entered content.
Input cleanup	`clean_input()`, `clean_message()`	Trims inputs and strips HTML tags.
Upload limits	Workflow functions	Limits uploads to 5 MB and mostly requires PDFs.
Audit trail	`log_audit()`	Provides accountability/history.
Transactions	Request creation, resubmission, decisions, registration	Prevents partial multi-table database operations.

Important improvements not implemented

- CSRF protection is ****NOT IMPLEMENTED****. POST forms have no CSRF tokens.
- Session ID regeneration after login is ****NOT IMPLEMENTED****.
- Secure/HttpOnly/SameSite session-cookie settings are not explicitly configured.
- Login throttling/account lockout is ****NOT IMPLEMENTED****.
- Password reset uses student number plus email and does not use a time-limited verification token.
- Admin password reset uses a known default password—unsafe in production.
- Database credentials are stored directly in `config.php`.
- Public upload-file URLs do not enforce authorization.
- Registrar upload uses extension validation only.
- There is no malware scanning.
- No HTTPS configuration is represented in source.
- There are no demonstrated automated tests.

Error types:

- Validation error: user input fails checks, e.g., invalid email or non-PDF upload.
- Authorization error: wrong role; redirect occurs.
- Database/system error: PDO exception; transaction rolls back where coded.
- Upload error: PHP upload error, excessive size, invalid MIME/extension, or failed move.

15. PDO concepts to explain

- PDO connection: `$pdo = new PDO(...)` connects PHP to MySQL.
- Prepared statement: SQL is prepared first, then values are supplied separately.
- `execute([$value])`: safely sends values to placeholders.
- `fetch()`: returns one database row.
- `fetchAll()`: returns multiple rows.
- `beginTransaction()`: begins an all-or-nothing database unit.
- `commit()`: permanently saves all transaction changes.
- `rollBack()`: cancels transaction changes after failure.
- `lastInsertId()`: gets the ID created by the latest insert, e.g., the new `request_id`.

Strong answer:

PDO was selected because it provides a clean database abstraction, prepared statements to reduce SQL injection risk, transaction support, and clear exception handling.

16. Frontend

HTML provides forms, tables, dashboards, navigation, approval controls, and document links.

CSS in `[style.css](C:/Users/K RICH/OneDrive/Music 2.0/SGCS/SGCS/assets/css/style.css)` provides:

- Responsive CSS Grid layouts.
- Cards and dashboard panels.
- Tables and status areas.
- Buttons/forms/alerts.
- A mobile media query at `max-width: 800px`.

JavaScript in `[main.js](C:/Users/K RICH/OneDrive/Music 2.0/SGCS/SGCS/assets/js/main.js)` only adds `role="alert"` to alert elements after page load.

JavaScript is not required for critical system operations. Login, uploads, approval, rejection, database updates, and workflow movement happen on the PHP server.

17. Ten code sections to know

File / section	Why it matters	Examiner question	Short answer
<code>config/database.php</code>		PDO connection	Why PDO? Secure prepared statements, transactions, and exception handling.
<code>includes/auth.php::require_role()</code>		Access control	How do you stop direct URL access? Every protected page calls a role guard before rendering.

`login.php` POST block	Login	How are passwords checked?	Query active user, then call `password_verify()`.
`student_clearance_request.php` POST block	Entry point	How does a student start clearance?	Uploads transcript and calls the workflow function.
`start_clearance_request()`	Atomic request creation	Why transaction?	It keeps request, document, notifications, and audit records consistent.
`update_clearance_decision()`	Workflow engine	How do stages remain sequential?	It compares `current_dept_id`, then moves to next `sequence_no`.
`resubmit_clearance_request()`	Recovery from rejection	What happens after rejection?	Corrected transcript is uploaded and request returns to rejecting office.
`officer_requests.php` query	Officer queue	Can an officer see all requests?	No, only pending requests at their assigned department.
`final_approval.php`	Final result	Is it a certificate/PDF?	No; it is a student-only view of database-backed Final Approval.
`admin_users.php`	Administration	What can an admin control?	Accounts, roles, account status, reset, soft delete, and new department assignment.

18. Likely examiner questions

General

Question: What problem does SGCS solve?

Short answer: It digitizes graduation clearance and gives students visibility into progress.

Refer to: [index.php](C:/Users/K RICH/OneDrive/Music 2.0/SGCS/SGCS/index.php), workflow functions.

Question: Why is the workflow sequential?

Short answer: Graduation clearance requires every relevant office to complete its review before final approval.

PHP

Question: Why did you choose PHP?

Short answer: It is suitable for a server-rendered web application, integrates naturally with Apache/XAMPP and MySQL, and supports sessions, file uploads, and PDO.

Question: Why did you centralize logic in clearance_functions.php?

Short answer: Shared rules such as progression and notifications should be reused rather than duplicated across pages.

Database

Question: Why separate approval_records from clearance_requests?

Short answer: A request is the overall process; approval records preserve one department-specific decision and comment for each stage.

Question: How do you enforce one final approval per request?

Short answer: final_approvals.request_id is UNIQUE.

Security

Question: How do you prevent SQL injection?

Short answer: Main database operations use PDO prepared statements and parameter binding.

Question: Is CSRF protection implemented?

Short answer: No. It is a future security improvement.

Workflow

Question: How does the system stop a student from skipping stages?

Short answer: The system changes current_dept_id one department at a time using the next sequence number; officers see only requests currently assigned to them.

Question: What happens after rejection?

Short answer: The request becomes Rejected, the comment is saved, the student is notified, then can upload a corrected transcript and resubmit to the rejecting department.

UI/UX

Question: Is the interface responsive?

Short answer: Yes. CSS changes multi-column grids to one column below 800 pixels and enables horizontal table scrolling.

Testing

Question: What testing is visible in this repository?

Short answer: There are no automated test files. The current project relies on manual functional testing and seeded demo records.

Limitations

Question: Does Finance automatically verify payment?

Short answer: No. Finance verification is manual; a fee statement may be uploaded and viewed.

19. Difficult questions: concise defense answers

18. **Why not Laravel?**

This project uses plain PHP to demonstrate core PHP, PDO, session, and SQL concepts directly. Laravel would be appropriate for a larger production system.

19. **Why MySQL?**

The workflow is relational: students, users, departments, requests, approvals, and notifications have clear relationships.

20. **How is the Registrar prevented from approving first?**

Registrar requests appear only when current_dept_id is the Registrar department.

21. **How is the next stage calculated?**

The code selects the next department with a higher sequence_no.

22. **Where are rejection comments stored?**

In approval_records.comments.

23. **How are Final Approval numbers generated?**

final_approvals.approval_number is an auto-increment primary key.

24. **Can a student alter Final Approval?**

No. The Final Approval page is student-only and displays data without edit controls.

25. **Can students upload a new transcript after workflow starts?**

Only while status is Pending or Rejected; otherwise the transcript is shown as locked.

26. **What occurs if transaction creation fails?**

The code rolls back database actions; request creation also attempts to remove the uploaded file.

27. **Are all office roles separate database roles?**

No. Library, Finance, Dean, and Store share officer; department assignment distinguishes them.

28. **How do users receive notifications?**

The application inserts rows into notifications; users see them inside the application.

29. **Does it send email or SMS?**

No. Notifications are in-app only.

30. **Does Library require a project upload?**

No. A project file can be uploaded and viewed, but the code does not make it mandatory.

31. **Does Finance calculate fees?**

No. There is no payment calculation or external payment integration.

32. **Does the system produce a PDF?**

No. Final Approval is an HTML view only.

33. **What is one upload-security weakness?**

Uploads are stored under publicly reachable paths, and direct file links are not separately authorized.

34. **How are deleted users handled?**

Through soft deletion: deleted_at is set and account status becomes Inactive.

35. **Can an inactive user log in?**

No. Login selects only account_status = 'Active' and deleted_at IS NULL.

36. ****Does the system have CSRF protection?****

No; it should be added for production.

37. ****What would you improve first for production?****

CSRF tokens, stronger password-reset flow, secure upload delivery, HTTPS/cookie settings, audit coverage, tests, and payment integration.

20. Genuine project limitations and future improvements

Category	Current limitation	Appropriate improvement
Security	No CSRF protection.	Add token validation to all state-changing forms.
Security	Public upload paths.	Store files outside web root and authorize downloads.
Security	Password reset is weak.	Use tokenized, expiring email verification.
Security	Known default password reset.	Require admin-set temporary password and forced change.
Workflow	Manual Finance review only.	Integrate official fee/payment data.
Workflow	Documents are not mandatory per stage.	Configure required documents and stage-specific validation.
Final Approval	No PDF or digital signature.	Generate secure PDF with verification code/QR.
Scalability	One officer per department.	Support multiple officers, queues, delegation, and load assignment.
Administration	No department-management UI.	Add department CRUD and workflow configuration.
Testing	No automated tests in repository.	Add unit, integration, and security tests.
Deployment	Local XAMPP URL/configuration.	Use environment variables, HTTPS, production server configuration, and backups.

21. Five-minute presentation script

My project is the Students Graduation Clearance System, abbreviated as SGCS. It is a web-based system designed to digitize graduation clearance for Tangaza University.

The problem addressed is that manual clearance can be slow, difficult to track, and dependent on students physically moving from one office to another. Students may not know which office has delayed their clearance, while staff may not have a central record of decisions.

In SGCS, a student logs in, uploads a final transcript, and starts a clearance request. The request moves through a fixed sequence: Institute Director, Library, Finance Office, Dean of Students, University Store, and finally the Academic Registrar. The Academic Programme Leader is not included in the implemented workflow.

Each departmental officer sees only the requests assigned to their department. They can approve or reject a request and add a comment. If approved, the system moves the request automatically to the next department. If

rejected, the system records the reason, informs the student through an in-app notification, and allows the student to upload a corrected transcript and resubmit to the office that rejected it.

The student dashboard shows the overall status, current department, departmental progress, documents, and notifications. The student can also upload supporting documents such as a Final Year Project or fee statement. These documents can be viewed by departmental officers during their review.

After every department approves, the Academic Registrar makes the final decision. When the Registrar approves, the request status becomes Final and the system creates a Final Approval record. The student can then view a Final Approval page showing an approval number, issue date, personal details, programme, and the final approval status.

Technically, the system uses PHP, MySQL, PDO, HTML, CSS, JavaScript, and XAMPP. PDO is used for database access with prepared statements. PHP sessions keep users logged in, and role-based access control ensures that students, officers, registrars, and administrators access only their appropriate pages.

The project also uses database transactions when creating requests, resubmitting requests, and saving decisions. This protects against partial database updates. The main limitations are that Finance verification is manual, Final Approval is not generated as a PDF, uploaded documents need stronger access control, and CSRF protection is not yet implemented. These are clear areas for future improvement.

22. Ten-minute technical defense outline

38. ****Architecture:**** PHP server-rendered application; browser submits forms; PHP runs business logic; PDO communicates with MySQL.
39. ****Authentication:**** `login.php` queries active users and uses `password\_verify`; session values retain identity.
40. ****Authorization:**** `require\_role()` gates every protected module.
41. ****Database:**** explain `students`, `users`, `departments`, `clearance\_requests`, `approval\_records`, documents, notifications, final approvals, and audit logs.
42. ****Workflow:**** explain `current\_dept\_id`, `sequence\_no`, `approval\_records`, and `update\_clearance\_decision()`.
43. ****Transactions:**** show why inserting a request, document, notification, and audit log must behave atomically.
44. ****Uploads:**** student uploads validate actual PDF MIME type and size; limitations in direct public access and registrar extension-only check.
45. ****Notifications:**** database-backed in-app messages.
46. ****Final Approval:**** Registrar's final action writes `final\_approvals`; student reads it in view-only form.
47. ****Security:**** password hashes, prepared statements, RBAC, escaping, validation; then openly state missing CSRF, session hardening, secure downloads, and stronger reset flow.